Riassunto completo per lo studio

Programmazione Concorrente e Distribuita | Moduli 1.3, 1.4, 1.5 | UNIBO, a.a. 2025/2026

Sintesi organizzata in italiano delle 168 pagine dei tre documenti: Process Interaction and Coordination (66 pagine), Elements of Concurrent Program Design (33 pagine), Visual Formalisms (69 pagine). Mantiene la terminologia, le definizioni, i meccanismi, le formule e gli esempi trattati nel riassunto preparato in chat. Per i diagrammi originali si indicano anche le pagine delle slide.

Indice dei contenuti

- Modulo 1.3 - Semafori, sincronizzazione, problemi classici, monitor e condition variables
- Modulo 1.4 - Analisi e progettazione concorrente, architetture e task asincroni
- Modulo 1.5 - Reti di Petri, Statecharts e Activity Diagrams
- Ripasso - confronti, formule e domande di autovalutazione

MODULO 1.3 — Process Interaction and Coordination

1. Meccanismi di coordinamento

I programmi concorrenti includono processi che operano sullo stesso sistema e possono accedere a risorse condivise. I due problemi fondamentali sono la mutua esclusione (un solo processo entra nella sezione critica) e la sincronizzazione (vincoli sull'ordine di esecuzione di operazioni di processi differenti). Gli algoritmi realizzati esclusivamente con istruzioni macchina sono possibili, ma di livello troppo basso e difficili da programmare correttamente. Si introducono quindi semafori e monitor come meccanismi e astrazioni di coordinamento.

2. Semafori: definizione e operazioni

Un semaforo, introdotto da Dijkstra nel 1968, è una primitiva generale. È una struttura S = (S.V, S.L): S.V è un intero maggiore o uguale a zero; S.L è l'insieme degli identificativi dei processi bloccati. L'inizializzazione è S = (k, {}), con k >= 0. Le operazioni wait(S), o P(S), e signal(S), o V(S), devono essere atomiche.

```
wait(S):
  atomic:
    if S.V > 0: S.V = S.V - 1
    else:
      S.L = S.L union {current_process}
      current_process.state = blocked

signal(S):
  atomic:
    if S.L is empty: S.V = S.V + 1
    else:
      q = choose(S.L)
      S.L = S.L - {q}
      q.state = ready
```

Con wait, se il valore è positivo viene decrementato, altrimenti il processo viene bloccato. Con signal, se non vi sono processi in attesa il valore aumenta; altrimenti uno dei processi bloccati diventa ready. Ready non implica esecuzione immediata: decide lo scheduler.

Invarianti

Se k è il valore iniziale, #signal(S) il numero di signal eseguiti e #wait(S) il numero di wait completati con successo, valgono: S.V >= 0 e S.V = k + #signal(S) - #wait(S). Un wait bloccato non è ancora considerato completato.

3. Varianti dei semafori

- Binary/mutex: valori 0 o 1, utilizzati tipicamente per proteggere una sezione critica.
- Counting/resource: valori interi non negativi per quantità di risorse disponibili.
- Event: inizializzati a zero; vengono usati per segnalare il verificarsi di eventi.
- Weak: i processi sospesi sono in un insieme; signal ne sceglie uno arbitrario.
- Strong: i sospesi sono mantenuti in una coda FIFO; nel modello delle slide garantiscono assenza di starvation per N processi. La distinzione riguarda la liveness, non la safety derivata dalle invarianti.
- Busy-wait: non utilizzano S.L; wait attende attivamente che S.V > 0 e poi lo decrementa atomicamente. Possono consumare CPU e non garantiscono libertà dalla starvation. Sono utili con poca contesa e attese brevissime su multiprocessore.

```
busy_wait(S):
  atomic:
```

```
    await(S.V > 0)
    S.V = S.V - 1
busy_signal(S):
  atomic: S.V = S.V + 1
```

4. Sezione critica con semafori

Un semaforo binario inizializzato a 1 funziona da lock per processi concorrenti:

```
semaphore mutex = 1
process P:
  forever:
    NCS
    wait(mutex)
    CS
    signal(mutex)
```

NCS è la sezione non critica, CS la sezione critica. Il primo wait porta il semaforo a zero; gli altri processi aspettano. Le slide mostrano un diagramma degli stati ridotto per due processi (pp. 14-15), includendo posizioni dei processi, valore del semaforo e lista dei bloccati. Per due processi viene discussa la correttezza: mutua esclusione e assenza di deadlock e starvation nel modello analizzato. La soluzione si estende a N processi, ma un semaforo weak non garantisce generalmente assenza di starvation.

5. Sincronizzazione di eventi e merge sort

Un event semaphore realizza dipendenze di precedenza senza imporre necessariamente mutua esclusione. Per ordinare due metà di una sequenza in parallelo e combinarle soltanto al termine di entrambi gli ordinamenti:

```
S1 = 0; S2 = 0
P: sort_left();  signal(S1)
Q: sort_right(); signal(S2)
R: wait(S1); wait(S2); merge()
```

Non è necessario ordinare temporalmente P e Q, ma il merge deve avvenire soltanto dopo entrambi.

6. Producer–Consumer e bounded buffer

Il producer crea dati e il consumer li utilizza. Nel trasferimento sincrono devono essere pronti entrambi; la comunicazione asincrona impiega un buffer per disaccoppiare temporalmente le attività, consentendo velocità diverse e un numero variabile di agenti. Esempi: tastiera/sistema operativo, browser/comunicazione, word processor/stampante, programma grafico/schermo.

Buffer illimitato

Con capacità illimitata, il consumatore deve soltanto attendere un elemento disponibile. Un semaforo availItems inizializzato a 0 conta le disponibilità; nel modello con inserimento/prelievo atomici corrisponde al numero di elementi disponibili, considerando appropriatamente le operazioni in corso.

```
availItems = 0
Producer: forever: item = produce(); append(buffer,item); signal(availItems)
Consumer: forever: wait(availItems); item = take(buffer); consume(item)
```

Buffer limitato

Con capacità N servono due condizioni: non prelevare da vuoto e non inserire da pieno. Si impiegano availItems = 0 e availPlaces = N, semafori complementari (split semaphores).

```
Producer:
  item = produce()
  wait(availPlaces)
  append(buffer,item)
  signal(availItems)
```

```
Consumer:
  wait(availItems)
  item = take(buffer)
  signal(availPlaces)
  consume(item)
```

La relazione concettuale è availItems + availPlaces = N negli stati coerenti, tenendo conto dei posti/elementi riservati da operazioni in corso. Con diversi produttori e consumatori, se append e take non sono atomiche, si deve proteggere anche il buffer con un mutex.

```
mutex = 1
Producer:
  item = produce(); wait(availPlaces); wait(mutex)
  append(buffer,item)
  signal(mutex); signal(availItems)
Consumer:
  wait(availItems); wait(mutex)
  item = take(buffer)
  signal(mutex); signal(availPlaces); consume(item)
```

I semafori di disponibilità regolano la sincronizzazione, mentre mutex assicura mutua esclusione. È importante non mantenere il mutex durante un'attesa per spazio o dati.

7. Dining Philosophers e deadlock

Cinque filosofi alternano il pensare e il mangiare; per mangiare ognuno deve avere entrambe le forchette vicine. Ogni forchetta è una risorsa esclusiva. L'algoritmo ingenuo acquisisce prima la forchetta sinistra e poi quella destra, ma se tutti acquisiscono contemporaneamente la sinistra nasce deadlock: nessuno può ottenere la seconda.

```
wait(fork[i]); wait(fork[(i+1) % N]); eat()
signal(fork[i]); signal(fork[(i+1) % N])
```

Deadlock: insieme di processi bloccati in attesa circolare di risorse/eventi producibili solo dagli altri processi bloccati. Le condizioni di Coffman necessarie sono mutual exclusion, hold and wait, no preemption, circular wait. Il fenomeno è anche descritto come deadly embrace. I database possono usare grafi delle attese per rilevare cicli e interrompere una transazione vittima; la JVM non fornisce un recupero automatico generale dei deadlock tra thread.

Soluzione con ticket

Un semaforo ticket inizializzato a N - 1 limita il numero di filosofi autorizzati a cercare le due forchette. Evita la situazione in cui tutti possiedono una sola forchetta e attendono la seconda.

```
ticket = N-1
wait(ticket); wait(left); wait(right)
eat()
signal(left); signal(right); signal(ticket)
```

Soluzione con ordinamento delle risorse

Si assegna alle forchette un ordine totale e tutti i processi le acquisiscono secondo tale ordine: si impediscono dipendenze circolari.

```
first=min(i,(i+1)%N); second=max(i,(i+1)%N)
wait(fork[first]); wait(fork[second]); eat()
signal(fork[first]); signal(fork[second])
```

8. Readers–Writers

Più reader possono leggere contemporaneamente; un writer richiede accesso esclusivo, sia rispetto agli altri writer sia rispetto ai reader. Le invarianti sono nr >= 0, nw in {0,1}, nr > 0 implica nw = 0 e nw = 1 implica nr = 0. Usare un solo mutex per tutto garantisce safety ma serializza inutilmente i reader.

```
nr = 0; mutexR = 1; rw = 1
Reader:
  wait(mutexR)
  if nr == 0: wait(rw)
  nr = nr + 1
  signal(mutexR)
  read()
  wait(mutexR)
  nr = nr - 1
  if nr == 0: signal(rw)
  signal(mutexR)
Writer:
  wait(rw); write(); signal(rw)
```

Il primo lettore blocca i writer, gli altri possono leggere in parallelo; l'ultimo lettore libera la risorsa. La soluzione descritta non garantisce equità: un flusso di reader può ritardare indefinitamente i writer.

9. Altri problemi classici

Le slide richiamano il Barber Shop Problem, il Cigarette Smokers Problem (Patil, 1971) e il Santa Claus Problem (Bansal, 2006). Nel problema dei fumatori ci sono tre fumatori, ognuno dotato illimitatamente di uno tra tabacco, carta e fiammiferi, e un agente che deposita gli altri due ingredienti; il fumatore corretto prepara la sigaretta e segnala il completamento. L'argomento di impossibilità di Patil presuppone precise restrizioni al codice dell'agente e all'uso di condizionali e array di semafori: non è un'impossibilità generale di qualsiasi soluzione con semafori.

10. Monitor

Introdotti da Brinch Hansen (1973) e generalizzati da Hoare (1974), i monitor offrono un'astrazione più alta rispetto ai semafori. Incapsulano stato condiviso, procedure di accesso e politica di coordinamento. Le variabili interne sono protette e i processi esterni usano le procedure pubbliche. Al massimo una procedura per monitor è attiva alla volta, quindi la mutua esclusione è implicita; monitor differenti possono procedere contemporaneamente.

```
monitor Counter:
  state count = 0
  procedure inc(): count = count + 1
  procedure getValue(): return count
```

L'assenza di race condition nell'accesso allo stato non implica automaticamente assenza di starvation, che dipende dalla politica di scheduling e dalle code.

11. Condition variables

Le condition variables sincronizzano i processi in base a predicati sullo stato interno di un monitor. waitC(cond) sospende il chiamante nella coda della condizione e rilascia il lock del monitor, così altri possono modificare lo stato; signalC(cond) risveglia il primo in attesa. Se non c'è nessuno, signalC non memorizza un permesso e non ha effetto.

- emptyC(cond): verifica la coda vuota; signalAllC(cond): risveglia tutti i sospesi.
- waitC(cond,rank): accoda con un rango; minrank(cond): restituisce il rango minimo.

Differenza essenziale: il semaforo può conservare segnali come permessi, mentre una condition variable non memorizza una segnalazione emessa in assenza di sospesi. La ripresa richiede anche di poter eseguire nel monitor.

12. Discipline di segnalazione

- Signal and Continue: prosegue il segnalante; il risvegliato entrerà dopo. La condizione potrebbe essere cambiata, perciò si ricontrolla con while (!predicato) waitC(cond).
- Signal and Wait: il risvegliato prende il controllo e il segnalante si sospende; nel modello delle slide può essere sufficiente verificare il predicato con if.
- Signal and Urgent Wait: il risvegliato prosegue subito e il segnalante entra in una coda urgente, con precedenza sui nuovi processi esterni. È la semantica associata ai monitor di Hoare.

Con E = esterni, S = segnalante, W = risvegliato, gli ordini di precedenza nelle slide sono: Signal and Continue E < W < S; Signal and Wait E = S < W; Signal and Urgent Wait E < S < W.

13. Monitor applicativi

SynchCell e semafori implementati con monitor

Una cella sincronizzata contiene valore, flag available e condizione isAvail. set(v) memorizza e notifica; get attende finché il valore è disponibile. Varianti con signalAllC o signalC a catena mostrano risvegli multipli. Un semaforo può essere implementato con contatore, condizione notZero e procedure wait/signal: una versione incrementa il contatore prima del risveglio; un’altra trasferisce direttamente il permesso al sospeso.

Bounded buffer come monitor

Il buffer monitor impiega condizioni notFull e notEmpty: put attende se è pieno, poi inserisce e segnala notEmpty; take attende se è vuoto, poi preleva e segnala notFull. Nella versione ad array circolare si usano first e last. Con una posizione lasciata inutilizzata: vuoto quando first = last; pieno quando (last + 1) mod MAX_ELEMS = first.

RWLock e resource manager

Un monitor RWLock contiene nr, nw e le condizioni okToRead e okToWrite, con procedure request_read, release_read, request_write, release_write. I predicati di attesa devono rispettare la disciplina di segnalazione. Un ResourceManager incapsula stato, allocazione, rilascio, politiche e condizioni di disponibilità.

Implementare monitor tramite semafori

Si usano un mutex per l’ingresso, un semaforo condsem e un contatore condcount per ogni condition. waitC rilascia il lock prima di sospendersi e lo riacquisisce alla ripresa nella disciplina Signal and Continue. In Signal and Wait il segnalante deve invece cedere il monitor e aspettare.

14. Componenti riutilizzabili

Monitor e semafori consentono di progettare latch, barrier, rendez-vous, message box, blackboard ed event service. Si tratta di componenti che incapsulano il coordinamento e possono essere riutilizzati nella progettazione di architetture concorrenti.

MODULO 1.4 — Elements of Concurrent Program Design

1. Analisi e design

La progettazione si articola in due fasi: analysis, per individuare task, scomposizione di dati e dipendenze, e design, per selezionare un'architettura concorrente, assegnare task agli agenti e progettare risorse condivise e mezzi di coordinamento.

Un task è una sequenza coerente di istruzioni o, a un livello più astratto, un'unità di lavoro/responsabilità assegnata a un agente. Le dipendenze possono essere temporali o relative a dati e risorse; la loro analisi permette di aumentare il parallelismo ed evitare costi eccessivi di coordinamento.

2. Decomposizione del problema

- Task decomposition: prima si identificano task indipendenti o quasi, poi i dati necessari. Adatta, ad esempio, a sistemi produttore-consumatore. Le dipendenze dovrebbero pesare poco sul tempo complessivo.
- Data decomposition: prima si suddividono i dati in unità elaborabili relativamente indipendentemente, poi si definiscono i task. Può scalare bene con più processori; esempio moltiplicazione di matrici.
- Dependency analysis: analizza precedenze, dipendenze su dati e risorse e opportunità di esecuzione concorrente; raggruppa e ordina i task per migliorare prestazioni.

3. Componenti attivi e passivi

Gli agenti sono componenti attivi con un thread di controllo, responsabilità e logica di esecuzione. I componenti passivi incapsulano risultati, risorse condivise e media di coordinamento; nella disciplina proposta sono monitor. Gli agenti non espongono un'interfaccia pubblica usata direttamente da altri agenti: le interazioni sono mediate dai passivi. Entità ibride vanno suddivise in componenti attive e passive separate.

4. Tre classi concettuali di parallelismo

Result parallelism

Organizza la concorrenza sulla forma del risultato finale: ogni agente costruisce una parte, mentre strutture condivise proteggono il prodotto in costruzione. È vicino alla data decomposition. Esempio casa: squadre costruiscono componenti differenti (fondamenta, pareti, tetto), rispettando le dipendenze.

Specialist parallelism

Organizza il sistema come rete di specialisti; ogni agente esegue un tipo di attività e collabora tramite message box, blackboard, event service o protocolli. Esempio casa: geometri, scavatori, carpentieri e addetti al tetto, che operano in fasi sovrapposte e possono realizzare pipeline.

Agenda parallelism

Organizza la concorrenza attorno a una sequenza o workflow di attività: molti worker possono contribuire alla stessa fase, e in alcune condizioni fasi differenti procedono contemporaneamente. Le strutture condivise gestiscono i dati tra le fasi. Esempio casa: team generalista impegnato nell'agenda fondamenta → struttura → tetto.

Le tre classi sono modi distinti di ragionare sul problema ma non sono esclusive: si possono mescolare e applicare ricorsivamente. Si consiglia di iniziare con la classe più naturale e ottimizzare metodicamente se le prestazioni non sono soddisfacenti.

5. Architettura Master–Workers

Un master decompone il problema in sottotask, inserisce le descrizioni in un bag of tasks, raccoglie e aggrega i risultati. Una popolazione eventualmente dinamica di worker preleva task, li esegue e restituisce risultati. Il bag può essere una blackboard o un bounded buffer; i risultati possono essere raccolti in un bag o in un monitor aggregatore. Esempio: quadratura numerica.

6. Task, Executor e programmazione asincrona

Un task è un'unità logica di lavoro separata dal thread che la eseguirà: concorrenza logica e concorrenza fisica non coincidono necessariamente. Nei framework Java Executor, il master invia un task con execute/submit e prosegue senza attendere; un thread dell'executor lo esegue in base alla politica del framework. Il recupero del risultato può usare un bag, un collector o un Future.

7. Future

Un Future viene restituito subito dopo la sottomissione di una computazione asincrona e ne rappresenta il risultato o stato. Può consentire polling dello stato, attesa bloccante del risultato, cancellazione e raccolta di eccezioni. Un programma basato su task e Future è asincrono, ma può eseguire attese bloccanti: la programmazione strettamente event-driven richiede invece di evitare il blocco del flusso di gestione eventi.

8. Virtual Threads e Structured Concurrency

I thread leggeri (user-level threads, fibers, green threads) danno una vista logica disaccoppiata dai thread del sistema operativo. Le slide richiamano Java Virtual Threads, presentati in preview nel JDK 19 con JEP 425 come risultato del progetto Loom. La structured concurrency tratta gruppi di task correlati in thread differenti come una sola unità logica, facilitando gestione errori, cancellazione, affidabilità e osservabilità.

9. Fork–Join

Variante ricorsiva Master–Workers: un task si divide (fork) in sottotask che possono operare in parallelo; una fase join ne raccoglie e combina i risultati. Può impiegare un bag of tasks o strutture di creazione e attesa. Esempi: Merge Sort e quadratura adattiva; è adatto a problemi divide-and-conquer.

10. Filter–Pipeline

Gli agenti sono collegati da pipe o bounded buffer e hanno ruoli Generator, Filter e Sink. Il generator crea dati, i filter elaborano il flusso, il sink raccoglie risultati. Diversi stadi possono lavorare contemporaneamente su dati differenti; esempio: una sequenza di filtri per elaborazione di immagini.

11. Shared-Space / Blackboard

Gli agenti interagiscono indirettamente tramite uno spazio condiviso con operazioni di inserimento, ricerca e prelievo. Il comportamento è data-driven e offre temporal uncoupling: produttore e consumatore non devono essere attivi simultaneamente. Riferimento storico: Tuple Spaces e Linda di Gelernter.

12. Announcer–Listeners

Un annunciatore produce eventi, i listener si registrano agli eventi di interesse e un event service distribuisce notifiche. È una coordinazione event-based che separa temporalmente gli agenti e può sostenere l'esecuzione concorrente di handler.

13. MVC e concorrenza

Una lettura concorrente di MVC distingue controller attivi che gestiscono il controllo, model passivi con lo stato e la sincronizzazione, e view con i componenti di interfaccia. Molti toolkit GUI si basano su un singolo thread e su componenti non thread-safe: non è sicuro che agenti concorrenti li aggiornino direttamente. L'Observer tradizionale è sincrono; si può riformulare la propagazione di cambiamenti come produttore-consumatore con coda di eventi, disaccoppiando produzione e gestione.

MODULO 1.5 — Visual Formalisms

I formalismi visuali descrivono rigorosamente comportamenti dinamici, struttura dei task e dipendenze nei sistemi concorrenti. Sono strumenti per requisiti, analisi, design e, se formalizzati, verifica. Il modulo introduce reti di Petri, Statecharts e Activity Diagrams.

PARTE A — Reti di Petri

1. Struttura e significato

Le reti di Petri (Carl Adam Petri, circa 1965) modellano il flusso di controllo e informazione, particolarmente in presenza di eventi asincroni e concorrenti con vincoli di precedenza, frequenza o compatibilità. Sono grafi bipartiti orientati con places (cerchi), transitions (barre) e archi diretti tra nodi di tipi differenti. Il grafo rappresenta la struttura statica.

I token sono punti neri collocati nei posti. Una rete con token è una marked Petri net. La distribuzione dei token, detta marking/marcatura, costituisce lo stato corrente.

2. Regole di firing e non determinismo

Una transizione è enabled quando tutti i suoi posti di input contengono i token richiesti. Il firing è atomico: consuma i token dai posti di input e produce token nei posti di output. Diverse transizioni possono essere abilitate contemporaneamente, provocando evoluzioni alternative e non determinismo. Una source transition non ha input e produce token; una sink transition non ha output e li consuma.

Con archi pesati, una transizione t è abilitata se ogni posto p di input ha almeno w(p,t) token, e il firing consuma/produce quantità pari ai pesi degli archi. Esempio: $2\ H_2 + O_2 \rightarrow 2\ H_2O$, con consumo di due token idrogeno, uno ossigeno e produzione di due token acqua (slide p. 10).

3. Interpretazioni ed esempi

Posti, transizioni e posti di output possono rappresentare rispettivamente: precondizioni / evento / postcondizioni; dati in ingresso / calcolo / dati in uscita; segnali / elaborazione / segnali; risorse richieste / task / risorse rilasciate; condizioni / clausola logica / conclusioni; buffer / processore / buffer.

- Allocazione risorse (p. 12): un dispositivo o processore viene occupato solo se è disponibile e c’è una richiesta; la transizione finale lo rende nuovamente libero.
- Vending machine (p. 13): i posti rappresentano crediti di 0, 5, 10, 15 e 20 centesimi; gli archi modellano deposito di monete ed erogazione delle caramelle.
- Data-flow (p. 16): per x = (a+b)/(a-b), somma e sottrazione sono indipendenti, la divisione richiede entrambe e il ramo a-b = 0 indica risultato indefinito.

4. Concorrenza, conflitto, confusion

Due eventi sono concorrenti se entrambi possono verificarsi in qualsiasi ordine senza conflitto. Sono in conflitto se l’uno esclude l’altro, ad esempio perché consumano lo stesso token. Una situazione in cui conflitto e concorrenza si mescolano è detta confusion. Una transizione con input da più attività realizza sincronizzazione: può scattare solo quando tutte le precondizioni sono disponibili (esempio p. 15).

5. Tempo, località, attività complesse e gerarchie

Le reti ordinarie non introducono un tempo assoluto: gli eventi sono descritti tramite un ordine parziale. L’abilitazione e il firing dipendono da marcature e cambiamenti locali. Le attività non atomiche possono essere raffinate con più transizioni che distinguono inizio, fasi intermedie e conclusione. Le gerarchie raggruppano sottoreti per consentire analisi a vari livelli di dettaglio.

6. Modelli di problemi concorrenti

- Mutua esclusione: un token rappresenta l'unica autorizzazione disponibile; entrare in CS lo consuma, uscire lo restituisce (p. 23).
- Sincronizzazione: token prodotti da una transizione abilitano attività che devono seguire.
- Comunicazione: il messaggio è un token generato all'invio e consumato alla ricezione.
- Semafori: il valore di un semaforo è rappresentato dal numero di token; wait consuma, signal produce.
- Producer–Consumer: posti rappresentano dati disponibili e, per il buffer limitato, spazi liberi.
- Readers–Writers: la rete mostra accessi di più lettori e condizione di esclusività dello scrittore (p. 28).
- Dining Philosophers: posti per forchette e stati di pensare/mangiare, transizioni per acquisizione e rilascio (p. 29).

7. Inhibitor arcs e limiti di decidibilità

Gli archi inibitori consentono di testare l'assenza di token in un posto: una transizione può essere abilitata quando il posto indicato è vuoto. Le reti con archi inibitori hanno espressività paragonabile alle macchine di Turing, con conseguenti limiti generali di decidibilità per alcune proprietà.

8. Modello formale

La struttura è descritta dalla tupla C = (P,T,I,O), con P insieme dei posti, T delle transizioni, I funzione degli input e O degli output. La marcatura μ associa a ogni posto un numero naturale di token: μ: P → N. Una rete marcata è M = (P,T,I,O,μ). Le proprietà strutturali dipendono dal grafo; le proprietà comportamentali dalla marcatura ed evoluzione.

Esempio delle slide (p. 33): P={p1,p2,p3,p4,p5}, T={t1,t2,t3,t4}. Input: I(t1)={p1}, I(t2)={p2,p3,p5}, I(t3)={p3}, I(t4)={p4}. Output: O(t1)={p2,p3,p5}, O(t2)={p5}, O(t3)={p4}, O(t4)={p2,p3}.

9. Semantica e reachability

La funzione parziale δ(μ,t) definisce la nuova marcatura dopo il firing di una transizione abilitata; se non lo è, δ non è definita. L'esecuzione procede mediante μ0 → μ1 → μ2 → ... fino a quando non sono più disponibili firing. La reachability indica che una marcatura può essere raggiunta dalla marcatura iniziale mediante una sequenza finita di transizioni. Il reachability set raccoglie tutte le marcature raggiungibili: serve a verificare stati vietati, deadlock e mutua esclusione.

10. Matrici delle reti di Petri

Per m transizioni e n posti: la matrice di input D− (m×n) rappresenta i token consumati; la matrice di output D+ (m×n) quelli prodotti. La matrice di incidenza è D = D+ − D−. Se T è un vettore riga delle transizioni (1×m) e M la marcatura riga (1×n), il cambiamento si esprime con M' = T · D + M. Il calcolo della variazione non sostituisce il test di abilitazione.

11. Proprietà delle reti

- Safety / safe net: ogni posto contiene al massimo un token in ogni marcatura raggiungibile (1-bounded).
- k-boundedness: nessun posto può contenere più di k token in marcature raggiungibili.
- Conservativeness: la somma dei token è conservata nel corso dell'esecuzione.
- Dead transition: dalla marcatura considerata non esiste sequenza futura che la renda abilitata.
- Potentially fireable: esiste almeno una sequenza futura che può abilitarla.
- Live transition: resta potenzialmente eseguibile da ogni marcatura raggiungibile. La proprietà non implica automaticamente che un particolare scheduling la faccia eseguire.

12. Estensioni e strumenti

Le Timed Petri Nets associano ritardi a posti o transizioni, deterministici o stocastici. Le High-Level Petri Nets arricchiscono i token con valori/tipi simbolici e le transizioni con regole computazionali: tra esse Coloured Petri Nets e Predicate/Transition Nets. Le slide menzionano strumenti PIPE 2 e TINA per editing, simulazione e analisi.

PARTE B — Statecharts

13. Sistemi reattivi e motivazione

Gli Statecharts (David Harel, 1987) descrivono sistemi reattivi che rispondono continuamente a eventi interni/esterni, come interfacce, protocolli, sistemi embedded, avionica e automazione. Sono alla base dei diagrammi di stato UML. A differenza di sistemi transformational, modellano un comportamento continuativo. Le macchine a stati piatte soffrono dell'esplosione combinatoria degli stati, mitigata tramite gerarchia e ortogonalità.

14. Stati e transizioni

Uno stato è rappresentato da un rettangolo arrotondato; una freccia rappresenta il passaggio tra stati. L'etichetta segue lo schema evento (condizione) / azione: un evento può provocare la transizione se la guardia è soddisfatta, e può generare un effetto.

15. Clustering e refinement: XOR

Il clustering è bottom-up: raggruppa stati con comportamento comune in un super-stato. Il refinement è top-down: raffina uno stato in sottostati. Nella decomposizione XOR il sistema si trova in uno solo dei sottostati alla volta: D = A XOR C. Zoom in/zoom out consentono di visualizzare o nascondere i dettagli.

16. Default e history

Uno stato composto può definire un sottostato di default al quale entrare. La notazione History H registra lo stato precedente del componente: rientrando, il sistema può riprendere dal sottostato attivo prima dell'uscita anziché tornare al default.

17. AND e ortogonalità

Uno stato AND contiene regioni concorrenti separate graficamente da linee tratteggiate. Il sistema si trova contemporaneamente in un sottostato per ciascuna regione. Se una regione ha 2 stati e l'altra 3, esistono $2\times3 = 6$ combinazioni, senza doverle disegnare tutte. In generale con regioni di ampiezza $n1,\ldots,nk$ il numero di combinazioni è il prodotto $n1\times\ldots\times nk$. Una transizione può modificare una regione senza modificare l'altra.

La sincronizzazione tra regioni è possibile: un singolo evento può produrre cambiamenti in più componenti, per esempio (B,F) --α--> (C,G). L'ortogonalità rappresenta quindi concorrenza senza richiedere indipendenza completa.

18. Azioni, attività e visualizzazione

Le azioni sono idealmente istantanee e possono generare eventi, modificare condizioni e interagire con altri componenti o con l'ambiente. Possono essere associate alle transizioni, all'ingresso (entry) o all'uscita (exit) dagli stati. Le attività, invece, hanno durata non nulla e vengono tipicamente associate agli stati. L'unclustering è una tecnica grafica per migliorare la leggibilità senza cambiare la semantica del modello.

19. Esempi e strumenti

Le slide includono Statecharts di ventilatore e telefonata, oltre a esempi di gerarchie, regioni ortogonali e sincronizzazioni. Statemate è un ambiente per realizzare e simulare specifiche eseguibili di sistemi embedded, analizzare scenari, verificare interazioni e supportare i test.

PARTE C — Activity Diagrams

20. Workflow e componenti

Gli Activity Diagrams di UML descrivono flussi operativi e processi, mettendo al centro le attività e il loro ordine. Diversamente dagli Statecharts, centrati su stati ed eventi, descrivono il passaggio di controllo e dati tra azioni e decisioni.

- Initial node: avvia il workflow; Final node: ne rappresenta il completamento.
- Action/Activity: passo di lavoro.
- Decision: scelta di un ramo sulla base di una condizione; Merge: ricongiunge alternative.
- Fork: attiva più rami simultanei; Join: attende/sincronizza i rami prima di proseguire.

Decision/Merge rappresentano alternative; Fork/Join rappresentano concorrenza e sincronizzazione. Le slide (p. 64) propongono un workflow con attività parallele, decision e merge, seguito da un join.

21. Esempi e notazioni aggiuntive

Nel processo d’ordine (p. 65) la ricezione attiva in parallelo preparazione/spedizione e fatturazione; la spedizione distingue un ramo rapido (overnight) e uno ordinario (regular delivery); i flussi devono ricongiungersi prima della conclusione. Le swimlanes raggruppano le attività per attore/componente/thread responsabile. I signals rappresentano ricezione, emissione e trigger temporali. Il passing objects mostra anche i dati trasferiti tra le attività, non solo il flusso di controllo.

RIPASSO FINALE

1. Strumenti di coordinamento a confronto

- Mutex: un solo processo può accedere alla risorsa condivisa.
- Counting semaphore: conta i permessi relativi a risorse multiple.
- Event semaphore: impone una relazione di precedenza.
- Monitor: stato e operazioni con mutua esclusione implicita.
- Condition variable: consente l'attesa su predicati del monitor, rilasciando il lock.
- Bounded buffer: comunicazione asincrona e disaccoppiamento produttori-consumatori.
- Future: rappresenta una computazione asincrona e il suo futuro risultato.
- Blackboard: scambio indiretto di informazioni mediante spazio condiviso.
- Event service: distribuzione di eventi da annunciatori ad ascoltatori.

2. Safety vs. liveness

Safety: un evento scorretto non deve mai avvenire (accesso concorrente illegale, prelievo da vuoto, doppia assegnazione della stessa risorsa). Liveness: il sistema deve poter progredire, evitando deadlock e starvation. Una soluzione può soddisfare safety ma non liveness, come il primo algoritmo dei filosofi a cena.

3. I tre formalismi visuali

- Reti di Petri: posti, transizioni, token e marcature; modellazione di risorse, concorrenza e vincoli di abilitazione.
- Statecharts: stati, eventi, decomposizione gerarchica XOR e ortogonalità AND; comportamento reattivo.
- Activity Diagrams: sequenza delle attività, scelte, fork/join, swimlanes e flusso di oggetti; workflow.

4. Formule indispensabili

- Semaforo: S.V >= 0; S.V = k + #signal(S) − #wait(S).
- Readers–Writers: nr >= 0; nw in {0,1}; nr > 0 implica nw = 0.
- Rete di Petri: C = (P,T,I,O); marcata: M = (P,T,I,O,μ).
- Incidenza: D = D+ − D−.
- Evoluzione marcatura: M' = T·D + M.
- Stati ortogonali: N = n1 × n2 × ... × nk.

5. Domande per l'autovalutazione

- 1. Differenza tra mutua esclusione e sincronizzazione.
- 2. Differenza tra semafori strong, weak e busy-wait.
- 3. Perché il mutex binario inizializzato a 1 protegge una CS?
- 4. Come funzionano i due semafori del bounded buffer?
- 5. Quali sono le quattro condizioni necessarie per il deadlock?
- 6. Perché il primo algoritmo dei filosofi può andare in deadlock?
- 7. Come consente letture parallele il protocollo Readers–Writers?
- 8. Perché waitC rilascia il monitor?
- 9. Differenza tra Signal and Continue e Signal and Wait.
- 10. Differenza tra task decomposition e data decomposition.
- 11. Differenza tra Result, Specialist e Agenda Parallelism.

- 12. Master–Workers vs. Fork–Join.
- 13. Future: perché asincrono non significa non bloccante?
- 14. Posti, transizioni e token: che cosa rappresentano?
- 15. Quando una transizione è enabled?
- 16. Boundedness, safety e liveness nelle reti di Petri.
- 17. Che cosa esprime la matrice di incidenza?
- 18. Differenza tra decomposizione XOR e AND.
- 19. Che cos’è un history state?
- 20. Differenza tra Decision/Merge e Fork/Join.

Nota per l’esame: oltre alle definizioni, occorre saper spiegare correttezza e failure modes degli algoritmi, riconoscere deadlock, calcolare marcature e abilitazioni delle reti di Petri e interpretare i diagrammi. Gli schemi originali sono nelle slide indicate.